



Hochschule Aalen
UNIVERSITY OF APPLIED SCIENCES

FESTO

Masterarbeit

Fakultät Optik & Mechatronik

Modellbasierte Funktionsentwicklung

von

Betreuer:

Prof. Dr.-Ing. Jürgen Baur (Hochschule Aalen)
Prof. Dr.-Ing. Fabian Holzwarth (Hochschule Aalen)
Dr. Sergio Delgado (Festo SE & Co. KG)

Steffen Sieder

Mechatronik / Systems Engineering Master
88273
3. Semester

Oberhausen 11
73098 Rechberghausen
+49 157 78321760
siederst@online.de

Zeitraum: April 2026 - Oktober 2026

Sperrvermerk

Die nachfolgende Masterarbeit beinhaltet vertrauliche und interne Daten der Festo SE & Co. KG Veröffentlichungen und Vervielfältigungen - auch nur auszugsweise - sind ohne ausdrückliche schriftliche Genehmigung des Unternehmens nicht gestattet.

Die Arbeit ist nur den Gutachtern sowie den Mitgliedern des Prüfungsausschusses zugänglich zu machen.

Eidesstattliche Erklärung

Ich erkläre hiermit, dass ich die vorliegende Arbeit selbstständig und ohne Benutzung anderer als der angegebenen Hilfsmittel angefertigt habe. Die aus fremden Quellen direkt oder indirekt übernommenen Gedanken sind als solche kenntlich gemacht.

Die Arbeit wurde bisher in gleicher oder ähnlicher Form keiner anderen Prüfungsbehörde vorgelegt und auch nicht veröffentlicht.

Esslingen den 18. Mai 2026
(Ort, Datum, Unterschrift)

Gender-Hinweis

In dieser Arbeit wird aus Gründen der besseren Lesbarkeit das generische Maskulinum verwendet. Weibliche und anderweitige Geschlechteridentitäten werden dabei ausdrücklich mitgemeint, soweit es für die Aussage erforderlich ist.

Zusammenfassung

Die vorliegende Arbeit befasst sich mit der modellbasierten Funktionsentwicklung für Akkugeräte. Ziel ist es, zukünftig Funktionen für Akkugeräte modellbasiert in Matlab zu entwickeln und diese über die Autocodegenerierung in die Firmware zu integrieren. Im Rahmen dieser Arbeit werden erste Funktionen in die Simulationsumgebung übertragen und der Grundstein für die modellbasierte Funktionsentwicklung gelegt. Zunächst wird eine Modellstruktur sowie ein Ablauf für die Code-Generierung erarbeitet. Anschließend werden erste Maschinenschutzfunktionen in Matlab modelliert, in der Simulationsumgebung getestet und unter Berücksichtigung der Anforderungen aus dem Lastenheft verifiziert. In weiteren Systemtests werden an einer realen akkubetriebenen Motorsäge die Funktionen validiert.

Die abschließende Speicher- und Laufzeitanalyse zeigt, dass die modellbasierten Funktionen eine geringe Erhöhung des Ressourcenbedarfs im Vergleich zu den manuell codierten Funktionen verursachen. Dies ist vor allem auf die Schnittstelle zwischen dem Autocode und der Basissoftware zurückzuführen. Die Ergebnisse dieser Arbeit bieten eine solide Grundlage für den Übergang von der manuellen Programmierung hin zur modellbasierten Entwicklung, hinsichtlich nicht-sicherheitsrelevanter Funktionen.

Inhaltsverzeichnis

Abbildungsverzeichnis	III
Tabellenverzeichnis	IV
Skriptverzeichnis	V
1 Einleitung	1
1.1 Das Unternehmen FESTO	1
1.2 Problemstellung	2
1.3 Zukunftsziel	3
1.4 Zielsetzung dieser Arbeit	3
1.5 Aufbau dieser Arbeit	3
2 Fazit und Ausblick	5
A Literaturverzeichnis	7
B Programmcode	8
C Abkürzungen Matlab-Modell	12

Abbildungsverzeichnis

Tabellenverzeichnis

C.1	Abkürzungsverzeichnis Matlab - Modell	12
-----	---	----

Skriptverzeichnis

B.1 Skript für die Autocodegenerierung und Nachbearbeitung	8
--	---

1 Einleitung

Das erste Kapitel beinhaltet eine Einführung in das Unternehmen Festo SE & Co. KG sowie deren Konzernbereich Llifetech"vorgestellt. Des Weiteren wird die zugrunde liegende Problemstellung und anschließend die Zielsetzung der Arbeit erläutert. Abschließend wird die Struktur der Arbeit aufgezeigt.

1.1 Das Unternehmen FESTO

Ihren Ursprung fand die Festo SE & Co. KG im Jahre 1925 durch die Gründer Albert Fezer und Gottlieb Stoll, welche ursprünglich die Herstellung von Maschinen zu Holzbearbeitung anstrebten. Der anfängliche Unternehmensname FFezer & Stollentwickelte sich sehr schnell zu der Abkürzung „FESTO“ einer Kombination aus den beiden Nachnamen der Gründer. FESTO ist ein weltweit führender Hersteller für Automationstechnologie, welcher vor allem durch die Einführung und Weiterentwicklung der Pneumatik bekannt geworden ist. Das Unternehmen hat sich von einem kleinen Betrieb zu einem globalen Marktführer mit mehr als 60 Landesgesellschaften und über 20.000 Mitarbeitenden entwickelt und ist heute ein Anbieter innovativer Automatisierungslösungen für unterschiedlichste Branchen wie die Elektronik- und Automobilindustrie, die Lebensmittelverarbeitung und Verpackung, die Biotechnologie und Pharmaindustrie sowie den Life-Science-Bereich. Das umfangreiche Produktportfolio reicht von Industrierobotern, pneumatischen und elektrischen Antrieben, Ventilen und Greifern bis hin zu Lösungen für Flüssigkeits- und Gashandlingssysteme sowie verschiedenen Sensoren.

Als Familienunternehmen wird FESTO heute in der dritten Generation geführt. Neben dem Geschäftsbereich der Äutomationpositioniert sich FESTO seit Jahren als Weltmarktführer im Bereich der technische Aus-und Weiterbildung mit dem Geschäftsbereich der „Didactic“, welcher ein breites Spektrum an Schulungen, Trainings oder auch Lernsysteme für technische Bildungseinrichtungen und Unternehmen anbietet. [1]

In der Business Unit „Lifetech“ werden smarte Lösungen für die Medizintechnik und Laborautomatisierung entwickelt.

Die ANDREAS STIHL AG & Co. KG, 1926 von Andreas Stihl gegründet, ist ein weltweit führender Hersteller von motorbetriebenen Geräten. Das Unternehmen hat sich von einem kleinen Einmannbetrieb zu einem globalen Marktführer entwickelt, der eine breite Palette von Produkten anbietet, darunter Motorsägen, Heckenscheren, Freischneider und

Laubbläser. Diese Geräte sind sowohl für den professionellen als auch den privaten Gebrauch konzipiert.

In den letzten Jahren hat STIHL verstärkt in Akkutechnologien investiert, um umweltfreundlichere und geräuschärmere Alternativen zu Benzingeräten zu bieten.

Als Familienunternehmen wird STIHL heute in der dritten Generation geführt.

Innovationen sind ein zentraler Bestandteil der Unternehmensstrategie. STIHL setzt stark auf Forschung und Entwicklung, um die Effizienz und Ergonomie seiner Produkte zu verbessern. Das Unternehmen produziert viele seiner Komponenten selbst und garantiert durch strenge Qualitätskontrollen eine hohe Langlebigkeit der Geräte. STIHL ist in über 160 Ländern tätig und produziert weltweit in sieben Ländern. Die Anzahl der weltweit Beschäftigten liegt bei ungefähr 19.800 Mitarbeitern. Die internationale Vernetzung garantiert die Qualität von STIHL und ist ein Teil der Erfolgsgeschichte.

Ein weiterer Fokus liegt auf dem Thema Nachhaltigkeit, sowohl in der Produktentwicklung als auch in der Produktion, insbesondere durch die Reduktion von Emissionen und dem Einsatz energieeffizienter Prozesse. Insgesamt steht STIHL für Qualität, Innovation und weltweite Marktführerschaft in der Forst- und Gartengerätetechnologie

1.2 Problemstellung

Aufgrund der kontinuierlich steigenden Anforderungen an moderne Produkte, insbesondere im Hinblick auf deren Leistungsfähigkeit und Funktionalität, wird die Integration immer umfangreicherer Funktionen notwendig. Diese zusätzlichen Funktionen müssen überwiegend softwareseitig umgesetzt werden, um die geforderte Flexibilität und Anpassbarkeit der Produkte zu gewährleisten. STIHL verfolgt im Bereich der Verbrennungsmotoren bereits seit längerer Zeit den Ansatz der modellbasierten Funktionsentwicklung. Dabei werden die erstellten Modelle ausschließlich dazu verwendet, Softwarecode (Autocode) zu generieren, der direkt auf den Steuergeräten zum Einsatz kommt. Dies hat sich als effizienter Weg erwiesen, um komplexe Funktionen sicher und konsistent zu implementieren.

Im Gegensatz dazu erfolgt die Softwareentwicklung für Akkuprodukte bislang überwiegend manuell. Die Softwarefunktionen werden von Hand codiert, was im Laufe der Zeit zu einem zunehmend umfangreichen und unübersichtlichen Code führt. Zudem erschwert diese Vorgehensweise die Transparenz der Funktionen und erhöht den Aufwand für notwendige Anpassungen und Erweiterungen. Die manuelle Programmierung ermöglicht allerdings eine optimierte Anpassung des Codes an die gewählte Hardware und kann darüber hinaus Vorteile hinsichtlich Laufzeit und Speicherbedarf bieten.

1.3 Zukunftsziel

Das angestrebte Ziel der Softwarearchitektur von STIHL im Hinblick auf die Entwicklung von Akkugeräten ist in ?? dargestellt. Die Funktionsblöcke „Peripherie“ und „Hardware-nahe Funktionen“ sollen weiterhin manuell in der Programmiersprache C implementiert werden. Im Gegensatz dazu ist vorgesehen, die in Blau dargestellten kundenrelevanten Funktionen mittels Autocodgenerierung in Matlab-Simulink zu realisieren. Hierzu müssen die entsprechenden Funktionen zunächst in Simulink modelliert werden. Der daraus generierte Autocode wird anschließend über eine geeignete Schnittstelle in die manuell entwickelte Basissoftware integriert.

1.4 Zielsetzung dieser Arbeit

Das Ziel dieser Arbeit besteht darin, die Grundlage für die modellbasierte Entwicklung von Kundenfunktionen für Akkuprodukte zu schaffen. Dazu sollen im ersten Schritt die bereits in C implementierten Maschinenschutzfunktionen (siehe ??) in Matlab-Simulink übertragen werden. Die Modellierung erfolgt unter Berücksichtigung der relevanten Anforderungen aus dem Lastenheft. Der bereits vorhandene C-Code kann dabei als Referenz für die Erstellung des Simulink-Modells herangezogen werden. Im Anschluss sollen die modellierten Funktionen in der Simulationsumgebung getestet und verifiziert werden, um die Funktionalität sicherzustellen.

Im darauffolgenden Schritt ist die Zielsetzung, aus dem in Matlab erstellten Modell, C-Code zu generieren und in die bestehende Basissoftware zu integrieren. Hierfür soll die bereits vorhandene Schnittstelle zwischen dem automatisch generierten Code und dem manuell codierten Basiscode verwendet und angepasst werden. Der neu erstellte Softwarecode soll auf der realen Hardware abschließend getestet und validiert werden, um die Funktionsfähigkeit unter realen Bedingungen zu überprüfen. Darüber hinaus ist es ein weiteres Ziel eine Analyse der Auswirkungen des automatisch generierten Codes im Vergleich zur manuellen Codierung hinsichtlich Speicherplatzbedarf und Laufzeit durchzuführen.

1.5 Aufbau dieser Arbeit

Die vorliegende Arbeit gliedert sich in vier Hauptkapitel:

- ?? stellt den Stand der Technik im Bereich der Entwicklung von Akkuprodukten dar.

Hierzu wird zunächst eine kurze Übersicht zur modellbasierten Systementwicklung gegeben. Im Anschluss werden der aktuelle Stand der Firmware, die derzeitige Entwicklungsumgebung in Matlab sowie die bei STIHL geltenden Modellierungsrichtlinien vorgestellt.

- ?? zeigt die Vorbereitungen für die Modellierung und Codegenerierung auf. Dabei werden die Rahmenbedingungen für das Matlab-Modell durch die Auswahl einer geeigneten Modellierungsmethode, die Beschreibung des Modellaufbaus sowie spezielle Einstellungen für die Autocodegenerierung festgelegt.
- ?? erläutert die Implementierung der einzelnen Maschinenfunktionen, ausgehend von der Funktionsbeschreibung und der Analyse des Lastenhefts bis hin zur Modellierung in Matlab. Dabei werden sowohl die im vorangegangenen Kapitel gewonnenen Erkenntnisse als auch die festgelegten Modellierungsrichtlinien berücksichtigt.
- ?? präsentiert die Testergebnisse sowohl in der Simulationsumgebung als auch an einer realen akkubetriebenen Motorsäge in Bezug auf die zuvor modellierten Maschinenschutzfunktionen. Darüber hinaus werden der Speicherbedarf und die Laufzeit des generierten Autocodes in Kombination mit der Basissoftware analysiert.

Abschließend erfolgen ein Fazit und Ausblick der vorliegenden Arbeit.

2 Fazit und Ausblick

Im Rahmen dieser Masterarbeit wurde die Grundlage für die modellbasierte Funktionentwicklung im Bereich der Akkuprodukte bei STIHL geschaffen. Durch eine sorgfältige Vorbereitung des Matlab-Modells, insbesondere hinsichtlich der Modellstruktur und der automatisierten Code-Generierung, konnte ein solides Fundament für die Arbeit gelegt werden. Mithilfe von Variant-Subsystemen wurde die bestehende modulare Firmware-Architektur erhalten und erweitert.

Eine umfassende Analyse des Lastenhefts sowie des bestehenden C-Codes bildete die Basis für die Modellierung und Implementierung der Maschinenschutzfunktionen. Mithilfe der Simulink-Test-Toolbox wurden die einzelnen Funktionen in einer isolierten Testumgebung unter Verwendung selbst definierter Testabläufe automatisiert geprüft. Durch die Verknüpfung der Anforderungen aus dem Lastenheft mit den definierten Testabläufen konnten die implementierten Funktionen automatisch verifiziert werden.

Der C-Code für die Maschinenschutzfunktionen wurden mittels automatisierter Codegenerierung durch den Embedded-Coder erstellt und in die bestehende Firmware integriert. Über eine Konfigurationsoption in der Header-Datei der jeweiligen Maschine ist es möglich, während der Erstellung des Maschinencodes zwischen den automatisch generierten Funktionen und den vorhandenen Funktionen der Basissoftware zu wechseln. Bei aktivierter Autocode-Option werden die entsprechenden Basissoftware-Funktionen deaktiviert und während des Kompilierens des gesamten Codes nicht berücksichtigt.

Im Rahmen nachfolgender Systemtests wurden die modellbasiert entwickelten Maschinenschutzfunktionen erfolgreich an einer akkubetriebenen Motorsäge geprüft und verifiziert. Eine abschließende Analyse des Speicherbedarfs und der Laufzeit bei aktiviertem und deaktiviertem Autocode veranschaulichte die Unterschiede zwischen dem Autocode und den manuell codierten Funktionen. Bei aktivierten Autocode-Funktionen erhöht sich der Speicherbedarf, primär bedingt durch die Schnittstelle zwischen der Basissoftware und dem Autocode, die für die Bereitstellung der Ein- und Ausgangssignale verantwortlich ist. Die Speichererhöhung bezogen auf den ROM beläuft sich lediglich auf 0,37 %, wenn der zusätzliche Speicherbedarf der Schnittstelle berücksichtigt wird, und auf 0,107 %, wenn ausschließlich die Funktionalität in Relation zum gesamten verfügbaren Speicher des Steuergeräts betrachtet wird. Auch die Laufzeit erhöht sich geringfügig aufgrund der zusätzlichen Abarbeitungszeit der Schnittstelle, was jedoch keinen signifikanten Einfluss auf die Gesamtperformance hat. Mit der zunehmenden Auslagerung von Funktionen aus der Basissoftware in den Playground relativiert sich der zusätzliche Speicherbedarf sowie die verlängerte Laufzeit der Schnittstelle.

Der modellbasierte Entwicklungsansatz führt im Vergleich zur manuellen Programmierung

zu einer geringfügigen Erhöhung des Speicherbedarfs und der Laufzeit. Jedoch überwiegen die Vorteile dieses Ansatzes. Die strukturierte Implementierung der Funktionen auf Modellebene verbessert deren Nachvollziehbarkeit und Verständlichkeit erheblich. Dadurch können präzise Anpassungen einzelner Funktionen effizient und mit geringem Aufwand realisiert werden.

Zur abschließenden Prüfung und Validierung der in dieser Arbeit entwickelten Maschinenschutzfunktionen auf der realen Hardware sollte die Maschine mit aktiviertem Playground den standardisierten Prüfablauf der Testing-Gruppe durchlaufen.

Die Fortführung des Themas der modellbasierten Funktionsentwicklung ermöglicht es, weitere Funktionen aus der bestehenden Basissoftware in das Matlab-Modell zu übertragen. Hierbei kann die erarbeitete Modellstruktur sowie die konfigurierte Autocodegenerierung verwendet werden. Darüber hinaus besteht die Möglichkeit, die bestehende Schnittstelle zwischen der Basissoftware und dem Autocode weiter zu verwenden, Anpassungen vorzunehmen oder bei Bedarf zu erweitern.

Zusätzliche Untersuchungen könnten sich auf die Speicherbelegung in Bezug auf die Autocode-Einstellungen und die Modellierung konzentrieren. Derzeit erfolgt die Signalübergabe zwischen den Maschinenschutzfunktionen im Autocode durch den Zugriff auf globale Variablen. Zur Optimierung des Speicherbedarfs sollte dieses Verfahren überprüft und gegebenenfalls angepasst werden, um den Speicherverbrauch weiter zu minimieren. Dadurch bestehen weiterhin Möglichkeiten, die Leistungsfähigkeit und Effizienz des Autocodes zu verbessern.

Parallel zu dieser Arbeit wird ein physikalisches Modell in Matlab entwickelt, welches das Gesamtverhalten von Akkugeräten simuliert. Dieses Modell soll sowohl die physikalischen Eigenschaften als auch die Softwarefunktionen enthalten. Die im Rahmen dieser Arbeit implementierten Maschinenschutzfunktionen, die in einem Referenced-Subsystem abgespeichert sind, lassen sich mühelos in das physikalische Modell integrieren. Dabei ist lediglich die Übergabe der Eingangssignale sowie die Verarbeitung der Ausgangssignale zu berücksichtigen.

Zusammengefasst wird deutlich, dass der modellbasierte Ansatz auch künftig vielfältige Einsatzmöglichkeiten bei STIHL im Bereich der Akkugeräte bietet und zusätzliche Vorteile bei der Produktentwicklung generiert.

A Literaturverzeichnis

- [1] Festo SE & Co. KG. *Unternehmensgeschichte*. 2026. URL: https://www.festo.com/de/de/e/ueber-festo/unternehmen/unternehmensgeschichte-id_34983/ (besucht am 16.05.2026).

B Programmcode

```
1 %% Wichtig zu wissen!
2 % Ab Matlab 2023 kann die Datentyp Benennung für die Autocode Generierung
3 % angepasst werden, damit diese direkt mit der Firmware übereinstimmt.
4 % https://de.mathworks.com/help/ecoder/ref/datatypereplacement.html
5
6 %%
7 % Clear command window
8 clc
9 % close all files
10 fclose('all');
11 % Clear work folder and remove old build artefacts first
12 if exist('slprj','dir')
13     rmdir ('slprj', 's');
14 end
15 if exist([bdroot '_ert_rtw'], 'dir')
16     rmdir ([bdroot '_ert_rtw'], 's');
17 end
18 if exist(bdroot,'dir')
19     rmdir (bdroot, 's');
20 end
21 % create output directory
22 mkdir (bdroot);
23 delete *.slxc
24
25 %% build section
26 fprintf('\n\n');
27 fprintf('-----\n');
28 fprintf(' Code-Generation --> CPS_Data-structure \n');
29 fprintf('-----\n');
30 % Disable Verbose build mode
31 set_param(bdroot,'RTWVerbose',false);
32 % Build the model
33 rtwbuild(bdroot, 'generateCodeOnly', true);
34 % Enable Verbose build mode
35 set_param(bdroot,'RTWVerbose',true);
36
37 %% PostProcessing: generate CPS-fragment and rework source files
```

```
38 fprintf('\n\n');
39 fprintf('-----\n');
40 fprintf(' Code-Generation --> CPS_Fragment / Postprocessing \n');
41 fprintf('-----\n');
42 run create_A2L_fragment.m
43 copyfile('A2L_Fragment.c',bdroot);
44 delete 'A2L_Fragment.c'
45 run create_CPS_fragment.m
46 copyfile('CPS_Fragment.c',bdroot);
47 delete 'CPS_Fragment.c'
48 % Call ASAP2Creator to create A2L-Template based on ASAP2_Fragment
49 copyfile ../../000_Tools/002_A2L/ASAP2_Creator/ASAP2Creator.ini
50 copyfile ../../000_Tools/002_A2L/ASAP2_Creator/Master.a2l
51 copyfile ../../000_Tools/002_A2L/ASAP2_Creator/ASAP2Merger.ini
52 copyfile ../../000_Tools/002_A2L/ASAP2_Creator/Interface.a2l
53 system('ASAP2Creator');
54 system('ASAP2Merger -PASAP2Merger.ini -SModelbased_playground_Template.a2l
    -MInterface.a2l -OModelbased_playground_Template.a2l');
55 delete ASAP2Creator.ini Master.a2l ASAP2Merger.ini Interface.a2l
56 % Definitions
57 % define build directory
58 build_dir = [bdroot '_ert_rtw\'];
59 % define ignored files
60 % ignore_files_h = [strcat(bdroot,"_types.h"), strcat(bdroot,"_private.h"),
    "Product_parameter.h","rtmodel.h", "rtwtypes.h"];
61 ignore_files_h = ["Product_parameter.h","rtmodel.h", "rtwtypes.h"];
62 ignore_files_c = "Product_parameter.c";
63 % get all auto generated .h and .c files
64 files_h = dir([build_dir '*.h']);
65 files_c = dir([build_dir '*.c']);
66 % get all important files and replace Embedded Coder header file by ZUSE
    header file with same content
67 for i = length(files_h):-1:1
68     if sum(ismember(ignore_files_h, files_h(i).name)) > 0
69         files_h(i) = [];
70     else
71         find_replace([build_dir files_h(i).name], '#include
            "rtwtypes.h"', '#include <stdint.h>', '#include <stdbool.h>');
72         copyfile('temp.c', [bdroot '\ ' files_h(i).name]);
73         delete 'temp.c'
```



```
74     end
75 end
76 for i = length(files_c):-1:1
77     if sum(ismember(ignore_files_c, files_c(i).name)) > 0
78         files_c(i) = [];
79     else
80         find_replace([build_dir files_c(i).name], '#include
            "rtwtypes.h"', '#include <stdint.h>', '#include <stdbool.h>');
81         copyfile('temp.c', [bdroot '\\' files_c(i).name]);
82         delete 'temp.c'
83     end
84 end
85
86 %% CMakeList
87 fprintf('\n\n');
88 fprintf('-----\n');
89 fprintf(' for CMakeList in alphabetical order \n');
90 fprintf('-----\n');
91 ls = {};
92 for i = 1:1:length(files_c)
93     ls{end+1} = files_c(i).name;
94 end
95
96 for i = 1:1:length(files_h)
97     ls{end+1} = files_h(i).name;
98 end
99 ls{end+1} = 'Playground.c';
100 ls{end+1} = 'Playground.h';
101 ls = sort(ls);
102 for i = 1:1:length(ls)
103     disp(ls(i))
104     %disp('/n')
105 end
106
107 %% Finished
108 fprintf('\n\n');
109 fprintf('-----\n');
110 fprintf(' Code-Generation finished \n');
111 fprintf('-----\n');
112
```

```
113 %% Functions
114 function find_replace(file, old_h, int_h, bool_h)
115     fidi = fopen(file,'r');
116     fido = fopen('temp.c','w');
117     try
118         while ~feof(fidi)
119             l = fgetl(fidi); % read line
120             if contains(l,old_h)
121                 l = replace(l, old_h, int_h); % modify header line
122                 fprintf(fido,'%s\n',l); % write new line with int header
123                 fprintf(fido,'%s\n',bool_h); % write extra bool header
124             else
125                 l = strrep(l,'float32_t','float'); % if present replace old
                    datatype with new (until Matlab 2023)
126                 l = strrep(l,'float64_t','double'); % if present replace old
                    datatype with new (until Matlab 2023)
127                 l = strrep(l,'bool_t','bool'); % if present replace old datatype
                    with new (until Matlab 2023)
128                 fprintf(fido,'%s\n',l); % write new line
129             end
130         end
131     catch
132         fclose(fido);
133     end
134     fclose(fidi);
135     fclose(fido);
136 end
```

Skript B.1: Skript für die Autocodegenerierung und Nachbearbeitung

C Abkürzungen Matlab-Modell

Abkürzung	Bezeichnung	Zusatzinfo	neu?
Akku	Akkumulator		Ja
Aktv	aktivieren, aktiv		Nein
Anm	Anlaufmodus		Nein
Aus	aus, ausschalten		Nein
Blk	blockieren, Blockade		Ja
Btr	Betrieb		Nein
bus	Bus-Signal	Spezieller Variablentyp	Nein
Dek	dekrementieren, Dekrement		Nein
dm	Drehmoment	physikalische Größe	Nein
Ein	ein, einschalten		Nein
Elk	Elektrik, Elektronik		Ja
Erk	Erkennung		Nein
flg	Flag	Spezieller Variablentyp	Nein
Ink	inkrementieren, Inkrement		Nein
Lim	Limit		Ja
lst	Leistung	physikalische Größe	Ja
Max	Maximal		Nein
Mot	Motor, Maschine		Nein
Mtst	Motorsteuerung		Nein
n	Drehzahl	physikalische Größe	Nein
Nsr	not Safety relevant		Ja
num	Nummer, Zahl, Ziffer, Anzahl	Spezieller Variablentyp	Nein
Ok	In Ordnung, Ok		Ja
Red	reduzieren, Reduzierung		Ja
spng	Spannung	physikalische Größe	Nein
Sr	Safety relevant		Ja
stm	Strom	physikalische Größe	Nein
Stst	Stillstand		Ja
t	Zeit	physikalische Größe	Nein
tmp	Temperatur	physikalische Größe	Nein
Zael	Zähler		Nein
Zn	zu niedrig		Ja

Tab. C.1: Abkürzungsverzeichnis Matlab - Modell